



# USENIX

THE ADVANCED COMPUTING  
SYSTEMS ASSOCIATION

## HEC: Equivalence Verification Checking for Code Transformation via Equality Saturation

Jiaqi Yin and Zhan Song, *University of Maryland, College Park*;  
Nicolas Bohm Agostini and Antonino Tumeo, *Pacific Northwest National Laboratory*;  
Cunxi Yu, *University of Maryland, College Park*

<https://www.usenix.org/conference/atc25/presentation/yin>

This paper is included in the Proceedings of the  
2025 USENIX Annual Technical Conference.

July 7–9, 2025 • Boston, MA, USA

ISBN 978-1-939133-48-9

Open access to the Proceedings of the  
2025 USENIX Annual Technical Conference  
is sponsored by



جامعة الملك عبد الله  
للعلوم والتقنية  
King Abdullah University of  
Science and Technology

# HEC: Equivalence Verification Checking for Code Transformation via Equality Saturation

Jiaqi Yin\*      Zhan Song\*      Nicolas Bohm Agostini<sup>†</sup>      Antonino Tumeo<sup>†</sup>      Cunxi Yu\*

*\*University of Maryland, College Park      <sup>†</sup>Pacific Northwest National Laboratory*  
*{jyin629,cunxiyu}@umd.edu*

## Abstract

In modern computing systems, compilation employs numerous optimization techniques to enhance code performance. Source-to-source code transformations, which include control flow and datapath transformations, have been widely used in High-Level Synthesis (HLS) and compiler optimization.

While researchers actively investigate methods to improve performance with source-to-source code transformations, they often overlook the significance of verifying their correctness. Current tools cannot provide a holistic verification of these transformations. This paper introduces HEC, a framework for equivalence checking that leverages the e-graph data structure to comprehensively verify functional equivalence between programs. HEC utilizes the MLIR as its frontend and integrates MLIR into the e-graph framework. Through the combination of dynamic and static e-graph rewriting, HEC facilitates the validation of comprehensive code transformations.

We demonstrate effectiveness of HEC on PolyBenchC benchmarks, successfully verifying loop unrolling, tiling, and fusion transformations. HEC processes over 100,000 lines of MLIR code in 40 minutes with predictable runtime scaling. Importantly, HEC identified two critical compilation errors in mlir-opt: loop boundary check errors causing unintended executions during unrolling, and memory read-after-write violations in loop fusion that alter program semantics. These findings demonstrate HEC practical value in detecting real-world compiler bugs and highlight the importance of formal verification in optimization pipelines.

## 1 Introduction

In the post-Moore era, the pressing need to enhance the runtime performance of computing systems has driven extensive research into optimization strategies. Source-to-source transformations, such as control flow and datapath transformations, have been widely used to advance computing efficiency. Control flow transformations optimize parameters like unrolling and tiling factors to enhance hardware parallelism and performance [3, 45, 55]. Furthermore, there is significant interest in datapath and gate-level transformations [11, 16, 24, 56].

These combined efforts highlight a multi-faceted approach to optimize computing efficiency in various areas, including HLS [14, 15, 27] and compiler optimization [26]. However, the stability and correctness of these transformations are often not guaranteed. It is essential to ensure the equivalence between the programs before and after transformations.

Existing verification approaches typically focus on a single aspect of verification, thus failing to provide a comprehensive framework for verifying multi-domain transformations. For example, Polycheck [7] and ISA [2] attempt to verify equivalence for affine problems, devoted to control flow transformations. Furthermore, Samuel et al. [18] have developed verification methods specifically for datapath transformations in RTL code. These verification tools are limited in scope, and unable to offer comprehensive equivalence checking for both control flow and datapath transformations simultaneously.

As an emerging technique in compiler optimization [12, 46], hardware design automation [9, 10, 16, 48, 49, 54, 58], and theorem proving [4, 5, 21, 23], e-graph can represent multiple equivalent expressions within a single graph data structure. This capability enables possibilities of validation between optimized code against its original version. In this work, we propose HEC, a formal equivalence checking framework based on e-graph rewriting. HEC is designed to provide a holistic verification of codes through equality saturation. HEC combines static rewriting rules for datapath transformations with dynamic rewriting capabilities for control flow transformations. The main contributions of HEC are summarized as follows:

- Introducing HEC, an e-graph based verification framework taking MLIR codes as inputs and performing equivalence checking. It simultaneously verifies control flow and datapath transformations, ensuring holistic equivalence checking of multi-domain transformations with equality saturation.
- Designing a hybrid ruleset to combine static rewriting rules with dynamic rewriting capabilities within the e-graph infrastructure, enabling the framework to adapt to runtime-dependent transformation parameters and accommodate diverse transformation patterns.

```

1 %av, %bv: memref<101xi1>
2
3 %true = arith.constant true
4 affine.for %arg1=0 to 101 {
5   %1=affine.load %av[%arg1]:memref<101xi1>
6   %2=affine.load %bv[%arg1]:memref<101xi1>
7   %3=arith.andi %1, %2:i1
8   %4=arith.xori %3, %true:i1
9 }

```

Listing 1: Baseline Code

```

1 %av, %bv: memref<101xi1>
2
3 affine.for %arg1=0 to 101 {
4   %true = arith.constant true
5   %1=affine.load %av[%arg1]:memref<101xi1>
6   %2=affine.load %bv[%arg1]:memref<101xi1>
7   %3=arith.andi %1, %2:i1
8   %4=arith.xori %3, %true:i1
9 }

```

Listing 2: Variant B

```

1 %av, %bv: memref<101xi1>
2
3 %true = arith.constant true
4 affine.for %arg1=0 to 101 {
5   %1=affine.load %av[%arg1]:memref<101xi1>
6   %2=affine.load %bv[%arg1]:memref<101xi1>
7   %3=arith.xori %1, %true:i1
8   %4=arith.xori %2, %true:i1
9   %5=arith.ori %3, %4:i1
10 }

```

Listing 3: Variant C

```

1 %av, %bv: memref<101xi1>
2
3 %true = arith.constant true
4 affine.for %arg1=0 to 101 step 3 {
5   affine.for %arg2= %arg1 to min (%arg1+3, 101) {
6     %1=affine.load %av[%arg2]:memref<101xi1>
7     %2=affine.load %bv[%arg2]:memref<101xi1>
8     %3=arith.andi %1, %2:i1
9     %4=arith.xori %3, %true:i1
10   }
11 }

```

Listing 4: Variant D

Figure 1: Variants B, C, and D achieve equivalence with the baseline (Listing 1) through loop hoisting, De Morgan’s laws, and loop tiling, respectively. This multifaceted transformation, encompassing both datapath and control flow aspects, highlights the challenges in code verification. HEC offers a hybrid, combined solution to address this challenge.

- Introducing a robust graph representation to model MLIR input code, encapsulating both control flow and datapath structures. This representation unifies variable renaming, loop decomposition, and operation tracking, serving as an interface for integration with the e-graph framework.
- Demonstrating the scalability and efficiency of HEC through comprehensive evaluations on PolyBenchC benchmarks. Specifically, HEC can process over 100,000 lines of MLIR code in 40 minutes.
- Identifying two types of compilation errors introduced by the MLIR compiler with `mlir-opt`, in PolyBenchC [40,50] benchmark suite.

## 2 Motivating Example

**Unified datapath/control flow verification:** Code transformation is a critical aspect of code optimization, encompassing both control flow transformations and data path transformations. Typical control flow transformations include loop unrolling, tiling, and fusion, etc [19, 42, 47, 59]. Additionally, numerous studies focus on optimizing the computation graph at the operator level for datapath optimization [16, 17, 24, 56]. For instance, Jia et al. [24] optimize deep neural network (DNN) computations through graph substitution. These code optimizations underscore the necessity for rigorous verification of code transformations. In this section, we illustrate the challenges associated with code transformation verification using the example presented in Figure 1.

Listing 1 presents the baseline code that iterates over two vectors, each of length 101. For each pair of elements *a* and *b*, the loop performs the  $\text{NAND}(a, b)$  operation. In Listing 3,

a datapath transformation is demonstrated using De Morgan’s law, resulting in the  $\text{OR}(a', b')$  operation. Listing 2 and Listing 1 are functionally equivalent through loop hoisting, where the variable `%true` is moved inside the loop body. Similarly, Listing 4 and Listing 1 are equivalent via loop tiling, where the loop on Line 4 in Listing 1 is split into two loops.

However, conventional verification frameworks, including PolyCheck [7] and ISA [2], focus on only one aspect of verification. A holistic verification approach is essential to ensure the correctness and reliability of code transformations across all optimization levels, including control flow and datapath transformations. Individually verifying each transformation layer may overlook complex dependencies and interactions that can lead to subtle bugs or functional discrepancies.

Fortunately, e-graphs [53] enable the simultaneous representation of multiple equivalent program expressions, effectively managing dependencies and interactions between control flow and datapath transformations. They support bidirectional rewriting rules, allowing transformations to be applied flexibly while maintaining equivalence across optimization levels. Additionally, e-graphs offer a compact representation of equivalence classes, reducing computational overhead and enhancing scalability for large codebases. These advantages collectively motivate the development of a novel e-graph based verification approach. In this paper, we introduce how the four variants in Figure 1 are integrated into the e-graph.

**Challenges:** Although e-graphs provide prominent merits for efficient verification, conventional term rewriting still poses challenges when integrating control flow transformations. For example, loop tiling in Listing 4 creates a new loop variable `%arg2`, and the loop body (lines 6 to 9) replaces all instances that originally consume `%arg1` with the newly created variable `%arg2`. Additionally, the tiling factor (e.g.,

3 in Listing 4) is unknown until the compilation of the e-graph. Consequently, it is challenging to develop a static term rewriting pattern that can universally represent all tiling scenarios across diverse input codes. To effectively incorporate code equivalence checking within the e-graph framework, the rewriting rules must also include information that is only determinable at runtime, such as the specific tiling factor, variable names generated during transformation, and loop bounds that depend on input parameters. In these complex scenarios, *dynamic rewriting* rules must be tailored to the specific characteristics and requirements of the input code. Static rules alone are insufficient to handle runtime-dependent factors, which can vary significantly across different codebases and transformation instances. By customizing dynamic rules, HEC can accurately capture and verify intricate control flow transformation patterns, ensuring robust and reliable code equivalence checking even in highly sophisticated code structures. A more detailed discussion of the presented challenges and dynamic rewriting is provided in Section 4.2.

To this end, we propose HEC, a formal equivalence verification framework utilizing e-graphs with a hybrid approach of static and dynamic rewriting. The integration of both static and dynamic rewriting rules within e-graph ensures robust verification of multi-domain source-to-source transformations. In the following sections, we demonstrate how the different variants from Listing 2 to Listing 4 are verified to be equivalent to Listing 1 using various approaches: HEC graph representation, static rewriting, and dynamic rewriting.

## 3 Background

### 3.1 Source-to-Source Equivalence Checking

Code equivalence checking is a critical challenge in software engineering, which involves confirming that two code segments, despite having different syntactic or structural forms, exhibit identical behaviors under all operational conditions. This verification ensures that transformations made for optimization, refactoring, or adaptation do not inadvertently alter the functional output or introduce undefined behavior.

Consider two programs,  $P_A$  and  $P_B$ , that operate on a common input set  $I$ .  $P_A$  produces a set of possible outputs  $O_A$ , while  $P_B$  yields an output set  $O_B$ , both corresponding to the input set  $I$ .  $P_A$  and  $P_B$  are deemed functionally equivalent if and only if their output sets are identical for all possible inputs in  $I$ . This relationship can be formally expressed as:

$$\forall I, \quad O_A(I) = O_B(I).$$

This equation asserts that for every input  $I$ , the output  $O_A(I)$  of program  $P_A$  must exactly match the output  $O_B(I)$  of program  $P_B$ , thereby affirming their functional equivalence.

Some studies attempt to establish program equivalence checking across various domains [2, 7, 25, 41, 43, 52]. Weilei

et al. [7] introduced PolyCheck, a system designed to verify the equivalence of affine programs and their transformations through dynamic analysis. However, the applicability of PolyCheck is fundamentally limited to affine transformations, thereby restricting its use in broader code transformations. Similarly, other frameworks, such as ISA [2], suffer from restricted transformation coverage. Generally, these frameworks are sensitive to the format of the input code, and none can comprehensively provide equivalence checking that spans data path and control flow transformations simultaneously.

### 3.2 Multi-Level Intermediate Representation

MLIR [29] (Multi-Level Intermediate Representation) is a compiler infrastructure within the LLVM [28] project. Developed to address the complexity of generating efficient code for diverse hardware targets, MLIR facilitates the co-design of high-level optimizations and hardware-specific transformations. By providing a unified interface for different levels of abstraction, from high-level algorithmic constructs down to low-level hardware instructions, MLIR supports various domain-specific optimizations. Its modular design allows developers to define custom dialects and transformations, making it highly adaptable to various computing paradigms.

Some studies focus on optimizing and transforming the MLIR infrastructure to enhance HLS and code generation. For instance, CIRCT [13] converts MLIR to RTL code, serving as an open-source HLS tool. Cheng et al. [11] propose a MLIR based code optimization explorer based on the e-graph to address the phase-ordering problem [32]. Tools such as HeteroCL [26] and ScaleHLS [57] transform MLIR into HLS C, while other projects like SODA [3] and POLSCA [61] convert it into LLVM IR. Despite these developments, there is a lack of frameworks capable of supporting transformation verification for MLIR.

### 3.3 Equality Saturation

An e-graph (equivalence graph) is a data structure that compactly represents a congruence relation over expressions [37, 38, 46]. Widely utilized in compiler optimizations, program analysis, and formal verification, e-graph efficiently manages large sets of equivalent expressions by sharing common sub-expressions, allowing it to store an exponential number of expressions in linear space.

An **e-graph**  $\mathcal{G}$  is formally defined as a tuple  $\mathcal{G} = (N, E, \phi)$ :

- $N$  is a finite set of nodes (also called **e-nodes**). E-nodes  $N$  represent individual expressions or sub-expressions.
- $E \subseteq N \times N$  is an equivalence relation on  $N$ , partitioning  $N$  into equivalence classes known as **e-classes**. E-classes are the equivalence classes induced by  $E$ . Nodes within the same e-class are considered equivalent under the equivalence relation  $E$ . Here  $E$  has the property of closure under

congruence [36, 37] which ensures that the equivalence relation is preserved under the application of operators.

- $\phi : N \rightarrow \Sigma_k \times N^k$  is a labeling function that maps each node to an operator and a list of child nodes, where:
  - $\Sigma_k$  is the set of operators of operands number  $k$ .
  - $N^k$  denotes an tuple of  $k$  child nodes from  $N$ .

E-graphs have been used to power the optimization technique called **equality saturation**, which is an optimization process that exhaustively applies a set of rewriting rules to an initial expression within an e-graph until no new equivalence can be added. Formally, the following are defined:

- $G = (N, E, \phi)$  denotes an e-graph,
- $R = \{(l_1, r_1), \dots, (l_m, r_m)\}$  is the rewriting ruleset,
- $g_0$  is the initial expression encoded in  $G_0$ .

The process involves three main steps. First, construct  $G_0$  from the initial expression  $g_0$ . Second, for each pair  $(l_i, r_i) \in R$ , identify matches within  $G_i$  and add the equivalences  $[l_i]_E \sim [r_i]_E$  to form  $G_{i+1}$  via term rewriting [22]. Finally, iterate until  $G_{i+1} = G_i$ , indicating that no further changes occur. A detailed example of e-graph exploration is shown in Figure 2.

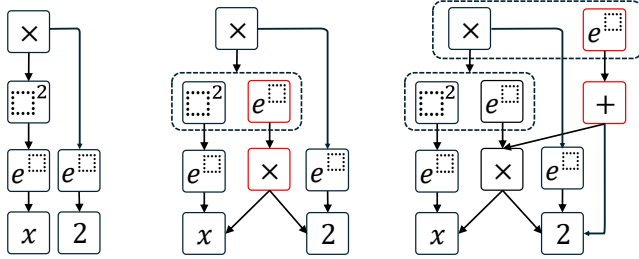


Figure 2: The e-graph rewriting for the expression  $(e^x)^2 \times e^2$ . **Left:** Initial e-graph. **Middle:** E-graph after applying the rewriting rule  $(e^x)^2 \times e^2 \Rightarrow e^{2x} \times e^2$ . **Right:** E-graph following the rewriting rule  $e^{2x} \times e^2 \Rightarrow e^{2x+2}$ . Dotted boxes represent e-classes; arrows show connections between e-nodes and their e-classes. Newly added e-nodes are highlighted in red.

Equality saturation has sparked research interests including compiler optimization [46], hardware design automation [16, 48, 54], theorem proving [4, 5, 21, 23], and more [8, 39, 44, 51, 56]. E-graph is supported by SMT solvers, such as Z3 [20, 21], and Willsey et al. developed an advanced e-graph framework **egg** for equality saturation [53, 60]. In this work, HEC incorporates the egg for e-graph implementation.

Formally, the functional equivalence of two programs  $P_A$  and  $P_B$  can be defined by representing programs within a shared e-graph  $G = (N, E, \phi)$  and verifying that their corresponding e-nodes  $n_A, n_B \in N$  reside within the same equivalence class. Specifically, the e-graph exhaustively applies a

comprehensive set of rewriting rules  $R$  to capture all possible code transformations. If, after saturation,  $n_A$  and  $n_B$  can be united in the same e-class, then programs  $P_A$  and  $P_B$  are deemed functionally equivalent. This approach transforms program equivalence checking into determining e-class membership within the e-graph, thereby ensuring that  $P_A$  and  $P_B$  exhibit identical behaviors across all inputs  $I$ .

## 4 Proposed Approach

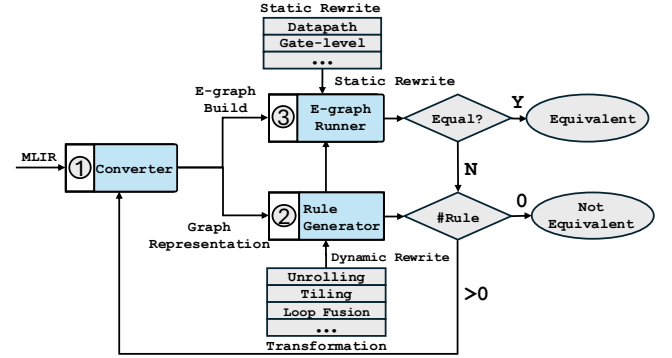


Figure 3: Overview of the HEC Framework: Initially, the input MLIR code is converted into a graph-based representation through dataflow analysis (step ①). This graph representation serves as a foundation for constructing the e-graph. The dynamic rules (step ②) are specifically designed to handle complex loop transformations that are not suitable for static rewriting. Finally, the e-graph runner (step ③) employs a hybrid ruleset that includes both static datapath rewriting and dynamic rewriting.

Here we provide an overview of HEC. To enhance versatility and avoid confinement to any specific programming language, HEC utilizes MLIR as its front-end input language. Figure 3 offers a comprehensive overview of the HEC approach. The core functionality of HEC lies in the verification of input code after datapath and control flow transformations, leveraging the e-graph with a hybrid static/dynamic rewriting ruleset.

HEC accepts MLIR as input language and any framework capable of generating MLIR code can serve as the front end, such as Polygeist [35] and IREE [1]. HEC incorporates a converter that processes the input MLIR code into a graph representation, acting as an interface for MLIR and the e-graph runner. Details of this graph representation are elaborated in Section 4.1, corresponding to step ① in Figure 3.

A major challenge within the HEC framework is to manage control flow transformations, such as loop unrolling and tiling. Standard e-graph rewriting techniques typically struggle to accurately represent these transformations. To address this issue, we have developed a dynamic rule generator (step ②) that creates rewriting rules at runtime to handle control flow transformations. This enables the creation of custom rewriting



rules tailored to each unique code input. The dynamic rule generation process is elaborated in Section 4.2.

Moreover, HEC incorporates static datapath rewriting rules. In Section 4.3, we present the e-graph based verification flow and the collaboration between static and dynamic rules, which are associated with step ③ in Figure 3. Additionally, we provide a detailed example to illustrate the process.

## 4.1 MLIR graph representation

Integrating the input MLIR code into the e-graph data structure requires translating the code into a graph representation. This translation process (step ①) involves two primary tasks to ensure the correctness of the mapping between the MLIR code and the graph representation. First, it needs to collect all necessary information to construct the e-graph, including dialects, operations, data type information, and other relevant metadata. Second, this involves explicitly representing loop transformation structures to ensure the graph accurately captures essential loop parameters like start, end, step, and the loop body. The generated graph representation functions as a bridge between MLIR, the e-graph runner, and the rule generator. Both the e-graph runner and the dynamic rule generator use the graph representation as input for further process.

**Task 1** – The essential information encapsulated in graph representation closely mirrors the structure and characteristics of MLIR operations. We designed this representation akin to an Abstract Syntax Tree (AST), where the graph nodes correspond to individual MLIR operations. Each node comprehensively encapsulates critical information, such as the operator’s name, input terms, data types, and dimensions. The interconnections between nodes are depicted as graph edges.

To construct the graph representation, we need to establish a one-to-one mapping between variable names and computation nodes. According to MLIR syntax, multiple variables may share the same variable name within their respective scopes. To address this issue, we rename all variables based on their global order of appearance in the input MLIR code. For example, if a variable %1 exists in multiple loop scopes, the variable in the subsequent loops will be renamed according to its appearance order. Similarly, all computation nodes are indexed based on their order of appearance. For example, the load operation in line 6 of Listing 1 is shown in Listing 5.

```

1 Vertex Name: Affine_Load_1
2 Dtype: i1
3 Dimension: 1
4 Input Edges: %bv, %arg1
5 Output Edges: %2
6
7 Edge Name: %2
8 Source Vertex: Affine_Load_1
9 Target Vertices: Arith_Andi_0

```

Listing 5: The graph representation attributes for the load operation and the term %2 in Listing 1 (line 6). In this example, the load operation is indexed based on its order of appearance, and is assigned an index of 1

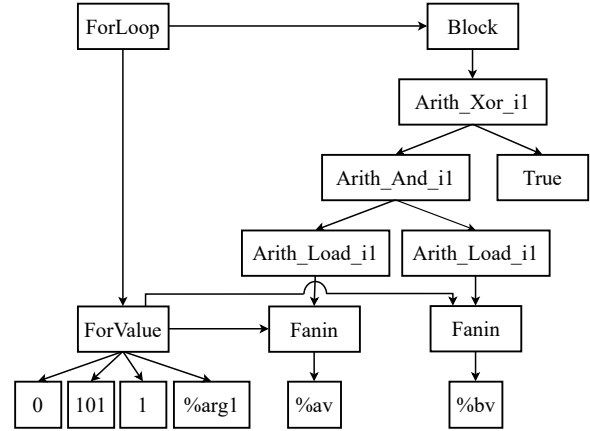


Figure 4: The dataflow graph for Listing 1

**Task 2** – The second challenge involves accurately representing *for* loop constructs within our graph representation. To address this, we decompose each *for* loop into two primary components: a *loopvalue* component and a *block* operation. A visualization of Listing 1 is provided in Figure 4. Specifically, the structure of a *for* loop is represented as follows:

- The *loopvalue* component encapsulates the loop parameters, including the loop start value, end value, and increment step. All operations that use the loop variable as input will consume the *loopvalue* component instead.
- To differentiate loop variables that share the same start, end, and step values, the *loopvalue* component includes the variable name as an additional input. This differentiation is not necessary for common datapath nodes such as *add*.
- The *block* operation represents the loop body. Specifically, the inputs to the *block* operation consist of the output terms from isolated operations performed within the loop body. Isolated output terms are those not consumed by any other operations within the block scope. For example, only %4 is isolated in Listing 1. The other output terms, from %1 to %3, are consumed by subsequent operations, thereby making %4 the only output for the *block* operation.
- For operations that do not produce output data, such as *affine.store*, HEC introduces a pseudo output term to maintain operation tracking. All pseudo output terms are considered isolated.
- The sequence of operations within the *block* operation is order-sensitive, ensuring that the original operation order is preserved within the loop block.
- All hierarchical structures, including *for*, *func*, and *if* constructs, maintain similar block operations.

Another benefit of such representation is its ability to automatically unify code transformations resulting from variable

Table 1: Selection of static rewriting rules integrating into HEC to manage datapath transformations. All datapath rewriting rules are signage and bitwidth dependent.

| Class      | Left-hand Side        | Right-hand Side                            |
|------------|-----------------------|--------------------------------------------|
| Datapath   | $a \ll b$             | $a \times 2^b$                             |
|            | $(a \times b) \ll c$  | $(a \ll c) \times b$                       |
|            | $a \times b \times c$ | $a \times (b \times c)$                    |
|            | $(a \ll b) \ll c$     | $a \ll (b + c)$                            |
| Gate-level | $\neg(a \& b)$        | $\neg a \parallel \neg b$                  |
|            | $a \oplus b$          | $(a \wedge \neg b) \vee (\neg a \wedge b)$ |
|            | $a \oplus 0$          | $a$                                        |
|            | $\neg a$              | $a \oplus \text{True}$                     |

movements without affecting data dependencies. For example, Listing 2 is transformed from Listing 1 through loop hoisting. In this transformation, the variable %true is moved inside the loop body. Since %true is consumed by the operation `arith.xori`, it is not isolated, and the block body does not track the term order. Both Listing 1 and Listing 2 correspond to the same dataflow depicted in Figure 4. The HEC graph representation automatically unifies the loop hoisting transformation before applying e-graph rewriting.

## 4.2 Hybrid E-graph Rewriting

In this section, we introduce the HEC hybrid rewriting rule sets, which encompass both static datapath rewriting rules and dynamic rewriting rules. We specifically focus on the dynamic rewriting rules, which are designed to handle control flow transformations.

**Static Rewriting:** HEC incorporates a comprehensive suite of static rewriting rules, including 62 bitwidth-dependent datapath rewriting rules. A selection of these static rewriting rules is presented in Table 1. These static rules are specifically designed to rewrite the input program at the operator level using Boolean algebra and standard arithmetic algebra, enabling efficient datapath transformations.

For example, as illustrated in Listing 3, the framework performs  $\text{OR}(a', b')$  operations for the input operands. Here, the NOT operation  $\text{NOT}(a)$  is expressed as  $\text{XOR}(a, \text{True})$  through the rewriting rule  $\neg a \Leftrightarrow a \oplus \text{True}$ . Similarly, the operations in Listing 1 form  $\text{NAND}(a, b)$ . These two code segments collectively constitute the left-hand side and right-hand side of the rewriting rule  $\overline{a \& b} \Leftrightarrow \overline{a} \parallel \overline{b}$ , after which the two code segments are unified into the same e-class.

**Why we need dynamic rewriting:** Pre-defined static rewriting rules provide robust support for datapath transformations, but they face significant challenges when applied to control flow transformations. To illustrate these challenges and to highlight the need for dynamic rewriting rules, we present a simple example.

```

1 %av, %bv: memref<101xi1>
2
3 %true = arith.constant true
4 affine.for %arg1 = 0 to 100 step 2 {
5 %1 = affine.load %av[%arg1] : memref<101xi1>
6 %2 = affine.load %bv[%arg1] : memref<101xi1>
7 %3 = arith.andi %1, %2 : i1
8 %4 = arith.xori %3, %true : i1
9 %5 = affine.apply affine_map<(d0) -> (d0 + 1)>(%arg1)
10 %6 = affine.load %av[%5] : memref<101xi1>
11 %7 = affine.load %bv[%5] : memref<101xi1>
12 %8 = arith.andi %6, %7 : i1
13 %9 = arith.xori %8, %true : i1
14 }
15 affine.for %arg2 = 100 to 101 {
16 %10 = affine.load %av[%arg2] : memref<101xi1>
17 %11 = affine.load %bv[%arg2] : memref<101xi1>
18 %12 = arith.andi %10, %11 : i1
19 %13 = arith.xori %12, %true : i1
20 }

```

Listing 6: Unrolling variant with Listing 1

```

1 (block
2 (forcontrol
3 (forvalue 0 101 1 %arg1)
4 (block
5 (arith.xori_i1 (arith.andi_i1 (load_i1 (fanin %av
6 (forvalue 0 101 1 %arg1))) (load_i1 (fanin
7 %bv (forvalue 0 101 1 %arg1)))) (
8 arith_constant_i1 1)))
9 )))

```

Listing 7: Rewriting rule lhs corresponding to Listing 1

```

1 #map = affine_map<(d0) -> (d0 + 1)>
2 (combine
3 (forcontrol
4 (forvalue 0 100 2 %arg1)
5 (block
6 (arith.xori_i1 (arith.andi_i1 (load_i1 (fanin %av
7 (forvalue 0 100 2 %arg1))) (load_i1 (fanin
8 %bv (forvalue 0 100 2 %arg1)))) (
9 arith_constant_i1 1)))
10 (arith.xori_i1 (arith.andi_i1 (load_i1 (fanin %av
11 (apply (map0 (forvalue 0 100 2 %arg1)))) (
12 load_i1 (fanin %bv (apply (map0 (forvalue 0
13 100 2 %arg1)))))) (arith_constant_i1 1)))
14 )))

```

Listing 8: Rewriting rule rhs corresponding to Listing 6

Consider the MLIR code shown in Listing 6, which is an unrolled version of Listing 1 with an unrolling factor of 2. To establish the equivalence between these two codes, the e-graph runner requires a rewriting rule of the form  $lhs \Leftrightarrow rhs$ , where the left-hand side (lhs) and right-hand side (rhs) are provided in Listings 7 and 8, respectively. However, constructing static rewriting rules to capture such transformations presents several challenges: (1) The number of operations within the *block* operation (line 5 in Listing 8) varies according to the unrolling factor. Static e-graph rewriting rules necessitate a predetermined number of input parameters prior to e-graph runner compilation, making it difficult to accommodate varying operation counts; (2) The isolated dataflow structures need to be isomorphic (line 5 in Listing 7 compared to lines 6, 7, and 12 in Listing 8). Assessing the isomorphism of these

Table 2: Selection of dynamic rewriting integrated into HEC to address control flow transformations. The pattern and transformation are interchangeable if they meet the conditions.

|            | Left-hand Side                                                                                                       | Right-hand Side                                                                                                                           | Condition                                                                                                                                                                            |
|------------|----------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Unrolling  | for <b>m1</b> to <b>n2</b> step <b>k2</b> :<br>Loop-body-2                                                           | for <b>m1</b> to <b>n1</b> step <b>k1</b> :<br>Loop-body-1<br>for <b>m2</b> to <b>n2</b> step <b>k2</b> :<br>Loop-body-2                  | 1. $\lceil (n2 - m1) / k2 \rceil == \lceil (n2 - m2) / k2 \rceil + \lceil (n1 - m1) / k1 \rceil \times (k1 / k2)$<br>2. Loop-body-1 is <b>k1/k2</b> times replication of Loop-body-2 |
| Tiling     | for <b>%1 = m1</b> to <b>n1</b> step <b>k2</b> :<br>Loop-body                                                        | for <b>%1 = m1</b> to <b>n1</b> step <b>k1</b> :<br>for <b>%2 = %1</b> to <b>n2</b> step <b>k2</b> :<br>Loop-body                         | 1. $k1 == f * k2$ ( $f$ is the tiling factor)<br>2. $n2 = \min(\%1 + k1, \%2)$                                                                                                       |
| Fusion     | for <b>m</b> to <b>n</b> step <b>k1</b> :<br>Loop-body-1<br>for <b>m</b> to <b>n</b> step <b>k2</b> :<br>Loop-body-2 | for <b>m</b> to <b>n</b> step <b>k1</b> $\times$ <b>k2</b> :<br>Loop-body-3                                                               | 1. Loop-body-3 is <b>k2</b> times replication of loop-body-1 plus <b>k1</b> times replication of loop-body-2<br>2. No memory RAW violation across Loop-body-1 and Loop-body-2        |
| Coalescing | for <b>%1 = m1</b> to <b>n1</b> step <b>k1</b> :<br>for <b>%2 = %1</b> to <b>n2</b> step <b>k2</b> :<br>Loop-body    | for <b>%3 = 0</b> to <b>n1</b> $\times$ <b>n2</b> :<br><b>%1</b> =(floordiv <b>%3 n2</b> )<br><b>%2</b> =(mod <b>%3 n2</b> )<br>Loop-body | 1. The reference of <b>%1</b> , <b>%2</b> is replaced by (floordiv <b>%3 n2</b> ) and (mod <b>%3 n2</b> ), respectively.                                                             |

dataflow structures is challenging simply using static rewriting rules; (3) Loop unrolling introduces new variables, and their metadata (such as variable names and data types) is collected at runtime. For example, the loop variable `%arg1` is created during the unrolling process in Listing 6. Since this runtime-collected information must be determined prior to compilation, creating static "unrolling" e-graph rewriting rules that accommodate all unrolling scenarios becomes impractical. In complex scenarios like loop unrolling, where the unrolling factor and variable metadata can vary dynamically based on input code, static rewriting rules prove insufficient. Therefore, dynamic rewriting rules must be customized to handle such variability. These challenges collectively prompt us to develop dynamic rewriting rules, generated at runtime, to effectively handle complex control flow transformations.

**Dynamic Rewriting:** To overcome the limitations of static rewriting, HEC generates dynamic rewriting rules at runtime, tailoring them to each unique code input to achieve high adaptability across diverse scenarios. In Step ②, the dynamic rule generator leverages the pre-defined transformation patterns to assess computation nodes for applicable transformations. Once suitable nodes are identified, HEC creates new dynamic rewriting rules based on the graph representation.

Using loop unrolling transformations as an example to demonstrate our verification approach, we establish the transformation criteria shown in Figure 5, which are used to determine whether a code segment qualifies as an unrolled variant. These criteria define the requirements for *for* loops to be recognized as equivalent to loop unrolling transformations. For instance, the loops in lines 4 and 15 of Listing 6 satisfy the conditions outlined in Figure 5, indicating their eligibility for the loop unrolling transformation pattern and confirming their equivalence to those in Listing 1. Additionally, we introduce

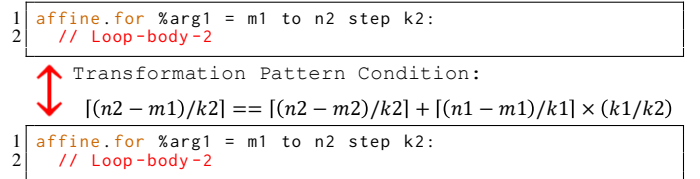


Figure 5: Loop unrolling transformation pattern: the code segments at the top and bottom can be interchanged if they satisfy the required conditions in the middle.

a leading operator, *combine*, to bind the two loop operations (line 2 in Listing 8). Here, we assume that all integrated rewriting rules and transformation patterns are correct. Whether the input code meets the transformation pattern conditions is automatically verified using the Z3 SMT solver [20, 21].

HEC analyzes the graph representation illustrated in Section 4.1 to assess the eligibility of each *for* loop. For each eligible loop, HEC constructs a dynamic rule  $lhs \Leftrightarrow rhs$ . For instance, Listing 7 and Listing 8 show the *lhs* and *rhs* of dynamic rewriting rules in Listing 6 unrolling, respectively. A selection of dynamic rewriting patterns is shown in Table 2.

Similarly, Listing 4 is verified to be equivalent to Listing 1 through dynamic rewriting. Specifically, the *for* loop in line 4 of Listing 4 is identified as an eligible computation node for loop tiling, and customized dynamic rewriting rules for tiling are established to achieve equivalence with Listing 1.

**Extensibility:** HEC supports extensibility through a systematic approach for incorporating new transformation patterns. For control flow transformations, users must formalize the transformation pattern along with its correctness conditions (similar to those in Table 2), while datapath transformations require the corresponding static rewriting rules. To ensure soundness, HEC employs a dual validation strategy: static dat-



apath transformation rules are derived from mathematically proven algebraic identities such as De Morgan’s laws and arithmetic associativity, making them sound by construction, while dynamic rules for control flow transformations undergo formal verification using the Z3 SMT solver to ensure mathematical correctness. For instance, the loop unrolling condition specified in Table 2 is mathematically verified to guarantee preservation of the iteration space, thereby maintaining semantic equivalence between the original and transformed code.

HEC limits Z3 usage to verifying dynamic rule pattern conditions rather than incorporating it into entire saturation process, thereby avoiding scalability bottlenecks. Our testing confirms that the dynamic rule generation, including Z3 checks, typically completes in under one second, a negligible fraction of the overall verification runtime, which is predominantly consumed by e-graph saturation.

### 4.3 HEC Verification Runner

```

1 %0 = arith.index_cast %arg0 : i32 to index
2 affine.for %arg2 = 0 to %0
3   %1 = affine.load %arg1[%arg2] : memref<?xf64>

1 #map1 = affine_map<() [s0] -> ((s0 floordiv 2) floordiv
  3) * 6)>
2 #map2 = affine_map<() [s0] -> ((s0 floordiv 2) * 2)>
3 #map3 = affine_map<() [s0] -> ((s0 floordiv 2) * 2 + ((s0
  mod 2) floordiv 3) * 3)>
4
5 affine.for %arg2 = 0 to #map1()[%0] step 6
6   %1 = affine.load %arg1[%arg2] : memref<?xf64>
7   %2 = affine.apply affine_map<(d0) -> (d0 + 1)>(%arg2)
8   %3 = affine.load %arg1[%2] : memref<?xf64>
9   %4 = affine.apply affine_map<(d0) -> (d0 + 2)>(%arg2)
10  %5 = affine.load %arg1[%4] : memref<?xf64>
11  %6 = affine.apply affine_map<(d0) -> (d0 + 1)>(%4)
12  %7 = affine.load %arg1[%6] : memref<?xf64>
13  %8 = affine.apply affine_map<(d0) -> (d0 + 4)>(%arg2)
14  %9 = affine.load %arg1[%8] : memref<?xf64>
15  %10 = affine.apply affine_map<(d0) -> (d0 + 1)>(%8)
16  %11 = affine.load %arg1[%10] : memref<?xf64>
17 affine.for %arg2 = #map2()[%0] to #map3()[%0] step 2
18   %1 = affine.load %arg1[%arg2] : memref<?xf64>
19   %2 = affine.apply affine_map<(d0) -> (d0 + 1)>(%arg2)
20   %3 = affine.load %arg1[%2] : memref<?xf64>
21 affine.for %arg2 = #map2()[%0] to #map3()[%0] step 3
22   %1 = affine.load %arg1[%arg2] : memref<?xf64>
23   %2 = affine.apply affine_map<(d0) -> (d0 + 1)>(%arg2)
24   %3 = affine.load %arg1[%2] : memref<?xf64>
25   %4 = affine.apply affine_map<(d0) -> (d0 + 2)>(%arg2)
26   %5 = affine.load %arg1[%4] : memref<?xf64>
27 affine.for %arg2 = #map3()[%0] to %0
28   %1 = affine.load %arg1[%arg2] : memref<?xf64>

```

Figure 6: MLIR Nested Unrolling Case Study: The bottom MLIR code is produced by applying nested unrolling with factors of 2 and 3 to the top MLIR code.

In this section, we will elaborate more details about e-graph verification runner and use a specific example to illustrate the verification process.

**Initial E-graph Construction:** HEC takes the graph representation as input. It gathers functional information and constructs the corresponding e-graph. The algorithm for e-graph construction is detailed in Algorithm 1. Before construction,

it builds the topological order for each operation based on the dependence relationship. Subsequently, HEC incrementally inserts operation nodes into the e-graph following the topological order of the graph nodes. Specifically, operations are inserted into the e-graph from the leaf nodes to the root nodes to ensure that child nodes are processed beforehand. Each insertion generates an identifier representing the e-nodes and connects these e-nodes with their respective input identifier.

#### Algorithm 1 E-graph Construction

---

**Input:** Vertex list  $V$  from parsing netlist  
**Output:** E-graph  $G_e$

- 1: Initialize  $vmap$  as empty HashMap
- 2: **for** each node  $v$  in  $TopoSort(V)$  **do** ▷ From leaf to root
- 3:    $in\_id \leftarrow []$
- 4:   **for** each input  $i$  for node  $v$  **do**
- 5:      $in\_id.pushback(vmap[i])$  ▷ All children are inserted
- 6:   **end for**
- 7:    $id \leftarrow G_e.insert(v, in\_id)$  ▷ Insert  $v$  to e-graph
- 8:    $vmap[v] = id$
- 9: **end for**

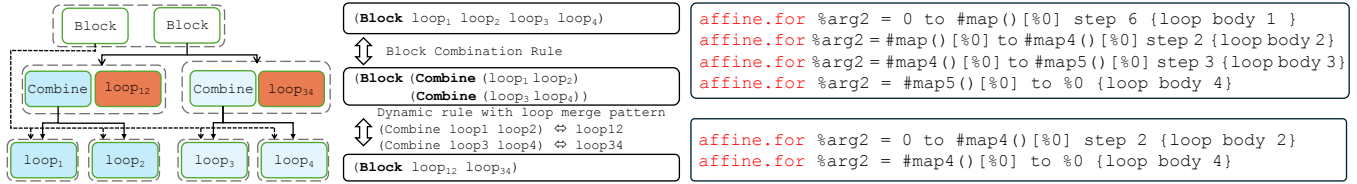
---

**HEC Verification:** To demonstrate the verification procedure, we use the example shown in Figure 6. The bottom MLIR code presents a nested loop unrolled with factors of 2 and 3, derived from the top MLIR code. The verification process is depicted in Figure 7. For simplicity, the *for* loop structures in Figure 4 are simplified to single-loop operations.

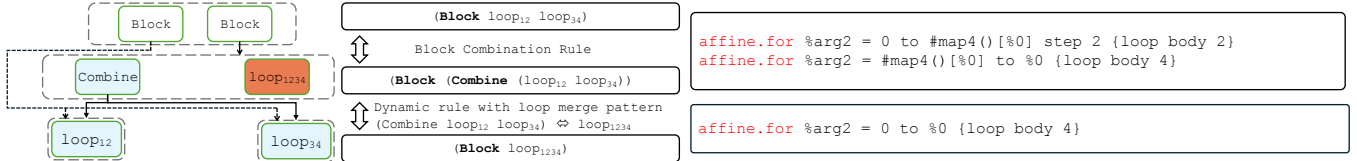
Initially, the ruleset includes only the predefined static rules without any dynamic rules. If the differences between the two MLIR codes fall within the scope addressable by static rewriting rules, the e-graph runner reports the codes as functionally equivalent. However, if the MLIR codes involve control flow transformations, the e-graph runner cannot affirm equivalence. In such cases, the graph representation is forwarded to the rule generator to apply the control flow transformation patterns.

Figure 7a illustrates the e-graph exploration during the initial iteration. The rule generator analyzes the graph representation and creates dynamic rules for candidate operations. HEC then creates a pseudo *combine* node to bind candidates for loop operations. The *combine* nodes are created automatically for each pair of candidate nodes of dynamic rules. Finally, HEC unifies the e-classes of *combine* node and the fused loop operations using dynamic rewriting rules. For example, the pairs of *for* loop structures from lines 5–17 and lines 21–27 in the unrolled MLIR code of Figure 6 (indicated by dotted lines in Figure 7a) are eligible for the loop unrolling transformation pattern. The e-graph runner creates a pseudo *combine* node as their parent e-node and builds dynamic rules to merge the *combine* node with *loop*<sub>12</sub> into the same e-class (indicated by solid lines). Consequently, *loop*<sub>1</sub>/*loop*<sub>2</sub> and *loop*<sub>3</sub>/*loop*<sub>4</sub> are merged into two loops named *loop*<sub>12</sub> and *loop*<sub>34</sub>, respectively. The MLIR codes for both the original loops and the unrolled version are displayed on the right side of Figure 7a.

However, the e-graph runner is still unable to ascertain the equivalence between the two input codes. An additional iteration is necessary for confirmation. Similar to the approach in



(a) In the first iteration, HEC inserted pseudo *Combine* nodes to bind leaf *loop*<sub>1</sub>/*loop*<sub>2</sub> and *loop*<sub>3</sub>/*loop*<sub>4</sub>. Then, the original sub-expression, indicated by a dotted line, is transformed into a new expression, represented by a solid line. Following this, the e-graph runner implements dynamic rules for *loop*<sub>1</sub>/*loop*<sub>2</sub> and *loop*<sub>3</sub>/*loop*<sub>4</sub> (highlighted by light blue nodes), merging them into *loop*<sub>12</sub> and *loop*<sub>34</sub> (illustrated with dark red nodes). However, HEC cannot verify the equivalence in the first iteration.



(b) In the second iteration, the rule generator introduces a new rule, merging *loop*<sub>12</sub> and *loop*<sub>34</sub> into *loop*<sub>1234</sub>, similar to the procedure in the initial iteration. Subsequently, the e-graph runner validates the equivalence between the two input MLIR codes.

Figure 7: E-graph verification for the motivating example provided in Figure 6. The e-graph operates through two iterations, with the rule generator introducing dynamic rules to the e-graph runner during each iteration. Each *loop* operator represents a *for* loop structure, as depicted in Figure 4, for brevity purposes.

the first iteration, the rule generator proposes a new dynamic rule for merging *loop*<sub>12</sub> with *loop*<sub>34</sub> into a single *loop*<sub>1234</sub>. The illustration of the second iteration is shown in Figure 7b. Consequently, the two MLIR codes in Figure 6 are unified into the same e-class. It is worth noting that Figure 7b shows only one possible verification path for clarity. In practice, the e-graph simultaneously explores multiple combination patterns, such as merging *loop*<sub>2</sub> and *loop*<sub>3</sub> into *loop*<sub>23</sub>, which could lead to alternative paths like *loop*<sub>1</sub>, *loop*<sub>2</sub>, *loop*<sub>3</sub>, *loop*<sub>4</sub> → *loop*<sub>1</sub>, *loop*<sub>23</sub>, *loop*<sub>4</sub> → *loop*<sub>123</sub>, *loop*<sub>4</sub> → *loop*<sub>1234</sub>. The e-graph’s congruence closure property ensures that all valid transformation sequences leading to equivalent programs are automatically discovered and unified within the same e-class. If the e-graph runner still cannot determine the equivalence and the rule generator fails to create new dynamic rules for further evaluation, HEC will conclude that the input MLIR codes are not equivalent. It is important to note that both the e-graph verification runner and the rule generator utilize the graph representation as their input. During each iteration, HEC is equipped with an inverter, which converts the e-graph back into the graph representation to facilitate further processing in the subsequent iteration. Besides, the verification flow is fully automated. HEC offers an interface for adding control flow transformation patterns, facilitating broader equivalence verification through an expanded set of rewriting rules and verification patterns.

**Completeness and Soundness:** HEC, like any verification system based on rewriting rules, is inherently incomplete with respect to the universe of all possible code transformations. If an equivalence exists but requires a transformation rule not included in our framework, or if the saturation process

is terminated due to computational limits before full exploration is complete, HEC may fail to establish equivalence even when it exists, resulting in false negatives (flagging equivalent programs as nonequivalent).

HEC guarantees soundness through a dual validation mechanism that ensures it will never produce false positives (declaring two programs equivalent when they are not). For static datapath transformation rules, soundness is achieved by deriving rules from mathematically proven algebraic identities such as De Morgan’s laws and arithmetic associativity, making them sound by construction. For dynamic control flow transformation rules, pattern conditions are formally verified using the Z3 SMT solver to ensure mathematical correctness.

## 5 Evaluation

The experiments were conducted on a system equipped with an Intel(R) Xeon(R) Gold 6418H CPU. This CPU has 48 physical cores, operates at a maximum speed of 4000 MHz, and the system has 1024 GB of RAM. We implemented transformations on selected benchmarks using the official MLIR compiler, *mlir-opt*, with LLVM version 18.0.0.

There are many existing transformation validation tools in the literature, including CompCert [30, 31], Alive/Alive2 [33, 34], PolyCheck [7], and MLIR-TV [6], each providing verification capabilities at different levels of the compilation stack. CompCert focuses on C-level compiler correctness, Alive/Alive2 target LLVM IR optimizations, PolyCheck handles affine code transformations for C-level compilation, while MLIR-TV is designed for MLIR-level verification. Among these tools, only MLIR-TV provides verification at

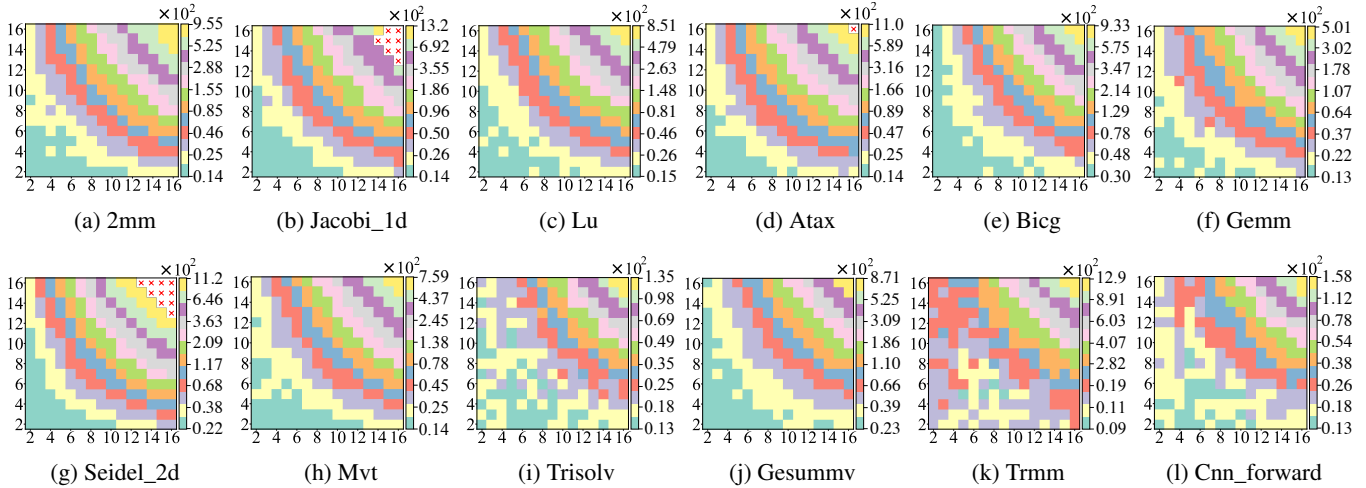


Figure 8: Heatmap of Runtime for Different Nested Unrolling Configurations. The x-axis and y-axis represent the unrolling factors, while the colorbar on the right illustrates the end-to-end verification runtime (seconds).

the MLIR level, which is most relevant to our work. However, according to our testing, MLIR-TV fails to support the affine dialect and lacks coverage for affine operations that are central to high-level synthesis and the transformations evaluated in our benchmarks. This limitation renders a direct comparison with HEC is challenging.

Table 3: List of benchmarks.

| Benchmark        | Description                            | Complexity       |
|------------------|----------------------------------------|------------------|
| GEMM [40]        | General Matrix Multiply                | $O(n^3)$         |
| LU [40]          | LU Decomposition                       | $O(n^3)$         |
| 2MM [40]         | Two Matrix Multiplications             | $O(n^3)$         |
| ATAX [40]        | Matrix Transpose Vector Multiplication | $O(n^2)$         |
| BiCG [40]        | Biconjugate Gradient Method            | $O(n^2)$         |
| GESUMMV [40]     | SUM of Matrix Vector Multiplications   | $O(n^2)$         |
| MVT [40]         | Matrix Vector Transpose                | $O(n^2)$         |
| TRISOLV [40]     | Triangular Solver                      | $O(n^2)$         |
| TRMM [40]        | Triangular Matrix Multiply             | $O(n^3)$         |
| CNN_Forward [50] | CNN Forward Function                   | $O(n^7)$         |
| Jacobi 1D [40]   | Jacobi 1D iterative method             | $O(n \cdot t)$   |
| Seidel 2D [40]   | Gauss-Seidel method                    | $O(n^2 \cdot t)$ |

## 5.1 Control Flow Transformation Verification

We evaluate the performance of HEC using the loop unrolling and tiling transformations on the MLIR benchmark. Specifically, we verify the equivalence between the input code within the selected benchmarks, reporting the end-to-end runtime, the number of dynamic rules implemented, and the number of e-classes. The benchmarks used in this evaluation are selected from Polybench [40] and are enhanced with an additional benchmark, CNN-Forward, from Polybench-NN [50]. We utilize Polygeist [35] to convert the C code into MLIR,

focusing specifically on the kernel portions for evaluation. These benchmarks are implemented as low-level kernels to facilitate hardware acceleration. The benchmarks, including description and time complexity, are shown in Table 3.

Comprehensive results are provided in Table 4, where all selected results have been successfully verified. The benchmarks for Seidel\_2d and Jacobi\_1d exhibit a specific type of error, classified as a **Loop Boundary Check Error**, which will be discussed in detail in Section 5.4.

First, HEC consistently verifies the correctness of transformations within one minute for most cases, except for nested unrolling. Second, the tiling transformation, which primarily adjusts loop parameters without altering code size, shows stable results across various tiling factors (represented as T2 to T64). Additionally, the number of e-classes has a significant impact on verification runtime, particularly in cases of nested unrolling. In these cases, the size of the e-graph predominantly determines the duration of verification.

## 5.2 Unrolling Verification Runtime

Since the unrolling transformation increases code size, unlike loop tiling, we specifically evaluate the verification runtime for nested unrolling as a scalability test. We assess the runtime performance of HEC across various nested unrolling factors. The results are illustrated in Figure 8, where each sub-graph is a heatmap. The x and y axes of the heatmap represent unrolling factors from 2 to 16. The color of each pixel indicates the verification runtime by seconds, as detailed by the colorbar on the right. Cases marked with an "X" indicate that they either exceeded the time limit.

Figure 8 shows that the verification runtime for most benchmarks completes within 20 minutes. E-graph saturation dominates the runtime, accounting for over 90% in runtime.

Table 4: Runtime, dynamic rule numbers, e-classes numbers comparison under various tiling (T), unrolling setup (U) configurations. For benchmark Jacobi\_1d and Seidel\_2d, mlir-opt tool generates the Loop Boundary errors, which will be elaborated in Section 5.4. Note that these are standard benchmarks used widely in MLIR-based compilation evaluations.

|             | Equivalence Checking                | Metric         | Base | T2~64 | U8   | U16  | U32  | U64  | T16-U8 | U16-T8 | U8-U4 | U16-U8 |
|-------------|-------------------------------------|----------------|------|-------|------|------|------|------|--------|--------|-------|--------|
| 2MM         | ✓                                   | Runtime(s)     | N/A  | 7.3   | 6.8  | 7.8  | 10.9 | 25.3 | 6.8    | 7.9    | 27.8  | 173.8  |
|             |                                     | #Dynamic Rules | N/A  | 2     | 2    | 2    | 2    | 2    | 4      | 4      | 6     | 6      |
|             |                                     | #E-classes     | 156  | 198   | 427  | 683  | 1195 | 2219 | 469    | 725    | 1733  | 5069   |
| Jacobi_1d   | <b>Loop Boundary Bug Identified</b> | Runtime(s)     | N/A  | 6.9   | 6.9  | 7.5  | 11.2 | 28.9 | 6.7    | 7.8    | 22.4  | 221.3  |
|             |                                     | #Dynamic Rules | N/A  | 1     | 2    | 2    | 2    | 2    | 3      | 3      | 6     | 6      |
|             |                                     | #E-classes     | 111  | 121   | 428  | 732  | 1340 | 2556 | 438    | 742    | 1832  | 5936   |
| Lu          | ✓                                   | Runtime(s)     | N/A  | 6.9   | 6.7  | 6.9  | 9.6  | 21.4 | 6.7    | 7.3    | 23.5  | 155.1  |
|             |                                     | #Dynamic Rules | N/A  | 1     | 2    | 2    | 2    | 2    | 3      | 3      | 6     | 6      |
|             |                                     | #E-classes     | 110  | 119   | 364  | 604  | 1084 | 2044 | 373    | 613    | 1642  | 4718   |
| Atax        | ✓                                   | Runtime(s)     | N/A  | 6.8   | 9.5  | 7.2  | 10.6 | 27.4 | 12.3   | 7.9    | 30.4  | 215.0  |
|             |                                     | #Dynamic Rules | N/A  | 2     | 3    | 3    | 3    | 3    | 6      | 4      | 9     | 9      |
|             |                                     | #E-classes     | 109  | 128   | 557  | 692  | 1252 | 2372 | 584    | 623    | 1930  | 5599   |
| Bicg        | ✓                                   | Runtime(s)     | N/A  | 22.2  | 20.1 | 21.8 | 24.0 | 37.8 | 25.8   | 25.9   | 40.6  | 191.3  |
|             |                                     | #Dynamic Rules | N/A  | 2     | 2    | 2    | 2    | 2    | 5      | 3      | 6     | 6      |
|             |                                     | #E-classes     | 139  | 158   | 513  | 666  | 1178 | 2202 | 540    | 597    | 1735  | 5143   |
| Gemm        | ✓                                   | Runtime(s)     | N/A  | 6.8   | 6.9  | 6.8  | 8.5  | 17.3 | 13.2   | 7.7    | 20.9  | 101.5  |
|             |                                     | #Dynamic Rules | N/A  | 1     | 2    | 2    | 2    | 2    | 3      | 3      | 6     | 6      |
|             |                                     | #E-classes     | 90   | 100   | 308  | 516  | 932  | 1764 | 318    | 526    | 1415  | 4078   |
| Seidel_2d   | <b>Loop Boundary Bug Identified</b> | Runtime(s)     | N/A  | 14.5  | 15.8 | 16.7 | 22.0 | 51.0 | 14.7   | 13.3   | 36.5  | 380.1  |
|             |                                     | #Dynamic Rules | N/A  | 1     | 1    | 1    | 1    | 1    | 2      | 1      | 3     | 3      |
|             |                                     | #E-classes     | 214  | 246   | 563  | 907  | 1595 | 3017 | 584    | 246    | 2125  | 6723   |
| Mvt         | ✓                                   | Runtime(s)     | N/A  | 6.8   | 6.3  | 6.9  | 9.2  | 20.6 | 6.1    | 6.0    | 25.7  | 144.7  |
|             |                                     | #Dynamic Rules | N/A  | 2     | 2    | 2    | 2    | 2    | 4      | 2      | 6     | 6      |
|             |                                     | #E-classes     | 91   | 131   | 339  | 579  | 1059 | 2019 | 356    | 131    | 1653  | 4683   |
| Trisolv     | ✓                                   | Runtime(s)     | N/A  | 6.8   | 7.2  | 6.3  | 7.0  | 9.3  | 6.2    | 6.7    | 17.7  | 36.0   |
|             |                                     | #Dynamic Rules | N/A  | 1     | 1    | 1    | 1    | 1    | 2      | 2      | 3     | 3      |
|             |                                     | #E-classes     | 82   | 92    | 214  | 334  | 574  | 1054 | 224    | 344    | 900   | 2400   |
| Gesummv     | ✓                                   | Runtime(s)     | N/A  | 19.5  | 19.1 | 19.3 | 21.4 | 31.1 | 17.8   | 18.3   | 30.3  | 131.5  |
|             |                                     | #Dynamic Rules | N/A  | 1     | 1    | 1    | 1    | 1    | 2      | 2      | 3     | 3      |
|             |                                     | #E-classes     | 172  | 181   | 402  | 618  | 1050 | 1914 | 411    | 627    | 1404  | 4372   |
| Trmm        | ✓                                   | Runtime(s)     | N/A  | 6.7   | 5.8  | 6.2  | 6.6  | 9.2  | 6.2    | 8.4    | 16.4  | 31.6   |
|             |                                     | #Dynamic Rules | N/A  | 1     | 1    | 1    | 1    | 1    | 2      | 2      | 3     | 3      |
|             |                                     | #E-classes     | 72   | 93    | 201  | 321  | 561  | 1041 | 222    | 342    | 896   | 2381   |
| CNN_forward | ✓                                   | Runtime(s)     | N/A  | 8.1   | 7.4  | 7.5  | 8.6  | 11.5 | 7.1    | 6.8    | 21.0  | 45.3   |
|             |                                     | #Dynamic Rules | N/A  | 1     | 1    | 1    | 1    | 1    | 2      | 1      | 3     | 3      |
|             |                                     | #E-classes     | 140  | 222   | 273  | 401  | 657  | 1169 | 343    | 222    | 983   | 2591   |



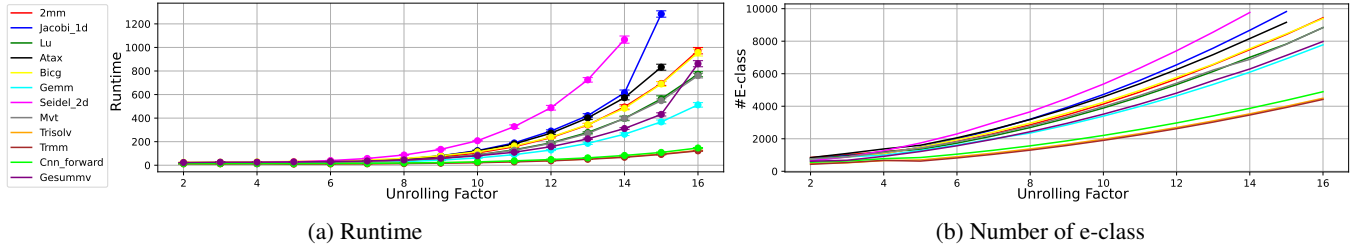


Figure 9: Runtime and e-graph complexity w.r.t nested unrolling complexity under the same unrolling factors.

Additionally, we present detailed results in Figure 9 for the diagonal data sample from Figure 8. In this figure, the x-axis denotes the nested unrolling data sample (specifically, the  $\text{unroll\_k\_unroll\_k}$ , where  $k$  is the unrolling factor), and the y-axis displays the runtime and the number of e-classes.

This runtime is exponential rather than linear. Firstly, unrolling transformations modify code size, introducing scalability challenges, which are not present in other transformations like tiling and loop fusion. Secondly, the diagonal data sample involves nested unrolling, where the code size increases quadratically with each increase in the unrolling factor. This quadratic growth in code size significantly impacts runtime. Notably, it is uncommon for implementations to adopt such large unrolling factors due to the substantial increase in chip area it entails. From the perspective of power, performance, and area (PPA), such high unrolling factors are generally avoided, making the case of nested unrolling particularly rare.

### 5.3 Datapath Transformation Evaluation

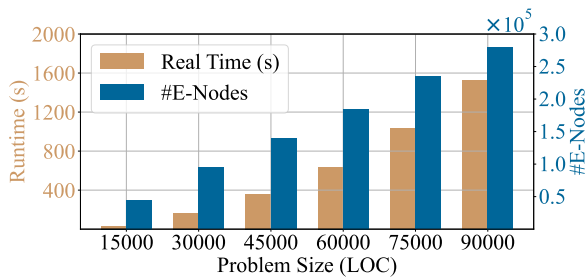


Figure 10: Runtime and e-nodes number for HEC across representative problem sizes ranging from 15,000 to 90,000 LOC.

HEC is designed to manage both static and dynamic transformations. In this section, we focus on the datapath transformation capabilities of HEC. Specifically, we generated over 150 benchmarks incorporating various datapath transformations. These benchmarks vary in size, measured by lines of code (LOC), to assess the framework scalability for datapath transformation. We conducted experiments to measure both the runtime and the number of e-nodes generated for different benchmark sizes. The results are illustrated in Figure 10. In this figure, the x-axis represents the problem size measured in the sum of LOC for input benchmark, with representative datapoints ranging from 15,000 to 90,000, while the y-axis depicts the runtime (left) and the number of e-nodes (right).

Notably, all verification tasks were successfully completed

within a 40-minute timeframe. For instance, the largest benchmark, encompassing 108,012 LOC, required a verification runtime of 2,305 seconds. This indicates that HEC maintains consistent performance even as the problem size increases significantly. Additionally, the analysis of the number of e-nodes reveals a linear growth trend with respect to LOC, as illustrated in Figure 10. This linear relationship suggests that the complexity of the e-graph structure scales predictably with the size of the input code, ensuring manageable computational overhead. Together, these findings highlight the capability of HEC to efficiently verify large-scale benchmarks while maintaining structural integrity and performance.

### 5.4 Mining Bugs in PolyBenchC MLIR

The MLIR affine dialect<sup>1</sup> provides a suite of standard optimization transformations. We selected the representative kernels to showcase the application of these transformations. We will highlight two representative cases where the optimized HLS programs fail to preserve semantic equivalence with the original programs, and our tool has successfully detected and reported these discrepancies as bugs. In the following sections, we will elaborate on these two cases as case studies.

**Case study 1: Loop Boundary Check Error:** We begin with an example in Listing 9 and its unrolled version (Listing 10), which is unrolled by a factor of 2 using `mlir-opt`. The two code segments appear equivalent. The first loop in Listing 10 from lines 9 to 13 handles the majority of the operations, while the second loop from lines 16 to 18 processes the remainder. Together, both loops make up the entire code shown in Listing 9. However, this transformation contains a flaw in the loop boundary checks. Specifically, if we consider the scenario where `%0` equals 5, Listing 9 will not execute since the loop start value (15) exceeds its end value (10). In contrast, Listing 10 executes once for the second loop spanning from line 16 to line 19. This error occurs with all values of `%0` less than 10.

Our tests indicate that these bugs are not confined to the specific case study presented. In general, for any loop defined with start and end conditions, `mlir-opt` preserves semantic equivalence only when the loop end value is guaranteed to exceed the loop start value. If this guarantee is not met and the loop is unrolled by a factor of  $k$ , the transformed loop may inadvertently alter the loop boundaries, causing unintended iterations. For instance, such errors are observed in the

<sup>1</sup><https://mlir.llvm.org/docs/Dialects/Affine/>



```

1 #map = affine_map<(d0) -> (d0 + 10)>
2 #map1 = affine_map<(d0) -> (d0 * 2)>
3 func.func @kernel(%arg0: i32, %arg1: memref<?xf64>) {
4   %0 = arith.index_cast %arg0 : i32 to index
5   // Loop range: from %0+10 to %0*2
6   // Program should not execute when %0 is 5 (15->10)
7   affine.for %arg2 = #map(%0) to #map1(%0) {
8     %1 = affine.load %arg1[%arg2] : memref<?xf64>
9   }
10  return
11 }

```

Listing 9: Case study 1

```

1 #map = affine_map<(d0) -> (d0 + 10)>
2 #map1 = affine_map<() [s0] -> (s0 + (s0 floordiv 2) * 2)>
3 #map2 = affine_map<(d0) -> (d0 + 1)>
4 #map3 = affine_map<(d0) -> (d0 * 2)>
5 func.func @kernel(%arg0: i32, %arg1: memref<?xf64>) {
6   %0 = arith.index_cast %arg0 : i32 to index
7   // Loop range: from %0+10 to %0+(%0 floordiv 2)*2
8   // This loop should not execute when %0 is 5 (15->9)
9   affine.for %arg2 = #map(%0) to #map1()[%0] step 2 {
10    %1 = affine.load %arg1[%arg2] : memref<?xf64>
11    %2 = affine.apply #map2(%arg2)
12    %3 = affine.load %arg1[%2] : memref<?xf64>
13  }
14  // Loop range: from %0+(%0 floordiv 2)*2 to %0*2
15  // This loop will execute once when %0 is 5 (9->10)
16  affine.for %arg2 = #map1()[%0] to #map3(%0) {
17    %1 = affine.load %arg1[%arg2] : memref<?xf64>
18  }
19  return
20 }

```

Listing 10: Loop boundary check error: Loop unrolling

Jacobi\_1d and Seidel\_2d benchmarks listed in Table 3.

**Case study 2: Memory Read-After-Write Violation:** Listing 11 and Listing 12 illustrate another instance of semantic non-equivalence following a transformation—loop fusion. In the provided code, the divergence in memory state between the two functions arises from differences in the execution order of the loop body operations after loop fusion. In the first function, two separate loops handle the operations: the first loop copies values from one element to the next, and the second loop increments these values. As a result, the final state of the array consists of the original element `%arg0[0]` followed by incremented versions of this element for all subsequent indices. Specifically, the array becomes `%arg0[0]`, `%arg0[0] + 1`, `%arg0[0] + 1`, ..., `%arg0[0] + 1`, with every element after the first set to `%arg0[0] + 1`. However, in the fused version, each iteration of the single loop performs both copy and increment operations adjacently. This introduces a dependency violation on the increment operation of the previous iteration, leading to a linearly increasing sequence based on the first element. Finally, the array becomes `%arg0[0]`, `%arg0[0] + 1`, `%arg0[0] + 2`, ..., `%arg0[0] + 10`. Such transformations in the loop structure affect not only the computational dependencies but also the overall memory state of `%arg0`, resulting in non-equivalence of the final output across the two function versions.

In these case studies, we identify two types of transformation errors introduced by the `mlir-opt` compiler. Code unrolling leads to additional loop body executions, while loop fusion transformation changes the sequence of data depen-

```

1 func.func @testing2(%arg0: memref<10xi32>, %arg1: memref
2   <10xi32>) {
3   %cst = arith.constant 1 : i32
4   // Replace elements indexed from 1 to 10 with %arg0[0]
5   affine.for %arg2 = 1 to 10 {
6     %1 = affine.load %arg0[%arg2 - 1] : memref<10xi32>
7     affine.store %1, %arg0[%arg2] : memref<10xi32>
8   }
9   // Elements indexed from 1 to 10 are incremented by 1
10  affine.for %arg2 = 1 to 10 {
11    %1 = affine.load %arg0[%arg2] : memref<10xi32>
12    %2 = arith.addi %1, %cst : i32
13    affine.store %2, %arg0[%arg2] : memref<10xi32>
14  }
15  // Finally, memory in %arg0 will be replaced to
16  // %arg0[0], %arg0[0]+1, %arg0[0]+1, ...
17  return

```

Listing 11: Case study 2

```

1 func.func @testing2(%arg0: memref<10xi32>, %arg1: memref
2   <10xi32>) {
3   %cst = arith.constant 1 : i32
4   affine.for %arg2 = 1 to 10 {
5     // Load data from %arg0[%arg2 - 1]
6     %0 = affine.load %arg0[%arg2 - 1] : memref<10xi32>
7     affine.store %0, %arg0[%arg2] : memref<10xi32>
8     %1 = affine.load %arg0[%arg2] : memref<10xi32>
9     %2 = arith.addi %1, %cst : i32
10    // Elements are incremented by 1
11    affine.store %2, %arg0[%arg2] : memref<10xi32>
12  }
13  // Finally, memory in %arg0 will be replaced to
14  // %arg0[0], %arg0[0]+1, %arg0[0]+2, ...
15  return

```

Listing 12: Memory RAW violation: Loop fusion

dencies, both affecting the intended behavior and memory state. Relying on `mlir-opt` for code transformations without subsequent equivalence checks can result in various adverse outcomes. Minor issues might include data corruption, while more severe implications could lead to complete system failures, potentially disrupting critical operations and compromising system integrity. Although code transformations can enhance performance, the improper application of these transformations without equivalence checking can introduce errors that disrupt control flow and alter semantic behavior.

## 6 Conclusions

This paper introduces HEC, a novel equivalence checking tool. HEC leverages the dynamic rule generation to effectively verify complex control flow transformations. By integrating both static datapath rewriting rules with dynamic, code-specific rules, HEC provides a flexible verification solution. Our experimental results demonstrate its efficiency, verifying various code transformations within an efficient runtime. Finally, HEC successfully identifies a few critical compilation errors, covering functional incorrectness and data hazard violations.

**Acknowledgment** – This work is supported by National Science Foundation (NSF) under CCF2350186, CCF2403134, CCF2349670, and CCF2349461 awards and the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research’s Advanced Computing Technologies Competitive Portfolios program at Pacific Northwest National Laboratory (PNNL).

## References

- [1] IREE, September 2019.
- [2] ISA. <https://repo.or.cz/w/isa.git>, 2024.
- [3] Nicolas Bohm Agostini, Serena Curzel, David Kaeli, and Antonino Tumeo. Soda-opt an mlir based flow for co-design and high-level synthesis. In *Proceedings of the 19th ACM International Conference on Computing Frontiers*, pages 201–202, 2022.
- [4] Leo Bachmair and Harald Ganzinger. Rewrite-based equational theorem proving with selection and simplification. *Journal of Logic and Computation*, 4(3):217–247, 1994.
- [5] Leo Bachmair and Harald Ganzinger. Equational reasoning in saturation-based theorem proving. *Automated deduction—a basis for applications*, 1:353–397, 1998.
- [6] Seongwon Bang, Seunghyeon Nam, Inwhan Chun, Ho Young Jhoo, and Juneyoung Lee. Smt-based translation validation for machine learning compiler. In *International Conference on Computer Aided Verification*, pages 386–407. Springer, 2022.
- [7] Wenlei Bao, Sriram Krishnamoorthy, Louis-Noël Pouchet, Fabrice Rastello, and Ponnuswamy Sadayappan. Polychex: Dynamic verification of iteration space transformations on affine programs. *ACM SIGPLAN Notices*, 51(1):539–554, 2016.
- [8] David Cao, Rose Kunkel, Chandrakana Nandi, Max Willsey, Zachary Tatlock, and Nadia Polikarpova. Babble: Learning better abstractions with e-graphs and anti-unification. *Proceedings of the ACM on Programming Languages*, 7(POPL):396–424, 2023.
- [9] Chen Chen, Guangyu HU, Cunxi Yu, Yuzhe Ma, and Hongce Zhang. E-morphic: Scalable equality saturation for structural exploration in logic~ synthesis. *Design Automation Conference (DAC)*, 2025.
- [10] Chen Chen, Guangyu Hu, Dongsheng Zuo, Cunxi Yu, Yuzhe Ma, and Hongce Zhang. E-syn: E-graph rewriting with technology-aware cost functions for logic synthesis. pages 1–6, 2024.
- [11] Jianyi Cheng, Samuel Coward, Lorenzo Chelini, Rafael Barbalho, and Theo Drane. Seer: Super-optimization explorer for hls using e-graph rewriting with mlir. *arXiv preprint arXiv:2308.07654*, 2023.
- [12] Jianyi Cheng, Samuel Coward, Lorenzo Chelini, Rafael Barbalho, and Theo Drane. Seer: Super-optimization explorer for high-level synthesis using e-graph rewriting. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 1029–1044, 2024.
- [13] Circuit ir compilers and tools (CIRCT)., 2024.
- [14] Jason Cong, Jason Lau, Gai Liu, Stephen Neuendorffer, Peichen Pan, Kees Vissers, and Zhiru Zhang. Fpga hls today: successes, challenges, and opportunities. *ACM Transactions on Reconfigurable Technology and Systems (TRETS)*, 15(4):1–42, 2022.
- [15] Jason Cong, Bin Liu, Stephen Neuendorffer, Juanjo Noguera, Kees Vissers, and Zhiru Zhang. High-level synthesis for fpgas: From prototyping to deployment. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 30(4):473–491, 2011.
- [16] Samuel Coward, George A Constantinides, and Theo Drane. Automatic datapath optimization using e-graphs. In *2022 IEEE 29th Symposium on Computer Arithmetic (ARITH)*, pages 43–50. IEEE, 2022.
- [17] Samuel Coward, George A Constantinides, and Theo Drane. Automating constraint-aware datapath optimization using e-graphs. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6. IEEE, 2023.
- [18] Samuel Coward, Emiliano Morini, Bryan Tan, Theo Drane, and George A Constantinides. Datapath verification via word-level e-graph rewriting. In *2023 Formal Methods in Computer-Aided Design (FMCAD)*, pages 92–100. IEEE, 2023.
- [19] Shail Dave, Youngbin Kim, Sasikanth Avancha, Kyoungwoo Lee, and Aviral Shrivastava. Dmazerunner: Executing perfectly nested loops on dataflow accelerators. *ACM Transactions on Embedded Computing Systems (TECS)*, 18(5s):1–27, 2019.
- [20] Leonardo De Moura and Nikolaj Bjørner. Efficient e-matching for smt solvers. In *Automated Deduction—CADE-21: 21st International Conference on Automated Deduction Bremen, Germany, July 17-20, 2007 Proceedings 21*, pages 183–198. Springer, 2007.
- [21] Leonardo De Moura and Nikolaj Bjørner. Z3: An efficient smt solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 337–340. Springer, 2008.
- [22] Nachum Dershowitz. A taste of rewrite systems. *Functional Programming, Concurrency, Simulation and Automated Reasoning: International Lecture Series 1991–1992 McMaster University, Hamilton, Ontario, Canada*, pages 199–228, 2005.

- [23] David Detlefs, Greg Nelson, and James B Saxe. Simplify: a theorem prover for program checking. *Journal of the ACM (JACM)*, 52(3):365–473, 2005.
- [24] Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. Taso: optimizing deep learning computation with automatic generation of graph substitutions. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, pages 47–62, 2019.
- [25] Chandan Karfa, Kunal Banerjee, Dipankar Sarkar, and Chittaranjan Mandal. Verification of loop and arithmetic transformations of array-intensive behaviors. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 32(11):1787–1800, 2013.
- [26] Yi-Hsiang Lai, Yuze Chi, Yuwei Hu, Jie Wang, Cody Hao Yu, Yuan Zhou, Jason Cong, and Zhiru Zhang. Heterocl: A multi-paradigm programming infrastructure for software-defined reconfigurable computing. In *Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, pages 242–251, 2019.
- [27] Yi-Hsiang Lai, Ecenur Ustun, Shaojie Xiang, Zhenman Fang, Hongbo Rong, and Zhiru Zhang. Programming and synthesis for software-defined fpga acceleration: status and future prospects. *ACM Transactions on Reconfigurable Technology and Systems (TRETS)*, 14(4):1–39, 2021.
- [28] Chris Lattner and Vikram Adve. LLVM: A compilation framework for lifelong program analysis & transformation. In *International symposium on code generation and optimization, 2004. CGO 2004.*, pages 75–86. IEEE, 2004.
- [29] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. Mlir: Scaling compiler infrastructure for domain specific computation. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 2–14. IEEE, 2021.
- [30] Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009.
- [31] Xavier Leroy, Sandrine Blazy, Daniel Kästner, Bernhard Schommer, Markus Pister, and Christian Ferdinand. Compcert-a formally verified optimizing compiler. In *ERTS 2016: Embedded Real Time Software and Systems, 8th European Congress*, 2016.
- [32] Bruce W Leverett, Roderic Geoffrey Galton Cattell, Steven O Hobbs, Joseph M Newcomer, Andrew Henry Reiner, Bruce R Schatz, and William A Wulf. An overview of the production quality compiler-compiler project. *Computer*, 13(8):38–49, 1980.
- [33] Nuno P Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. Alive2: bounded translation validation for llvm. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, pages 65–79, 2021.
- [34] Nuno P Lopes, David Menendez, Santosh Nagarakatte, and John Regehr. Provably correct peephole optimizations with alive. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 22–32, 2015.
- [35] William S Moses, Lorenzo Chelini, Ruizhe Zhao, and Oleksandr Zinenko. Polygeist: Raising c to polyhedral mlir. In *2021 30th International Conference on Parallel Architectures and Compilation Techniques (PACT)*, pages 45–59. IEEE, 2021.
- [36] Charles Gregory Nelson. *Techniques for program verification*. Stanford University, 1980.
- [37] Greg Nelson and Derek C Oppen. Fast decision procedures based on congruence closure. *Journal of the ACM (JACM)*, 27(2):356–364, 1980.
- [38] Robert Nieuwenhuis and Albert Oliveras. Proof-producing congruence closure. In *International Conference on Rewriting Techniques and Applications*, pages 453–468. Springer, 2005.
- [39] Pavel Panchekha, Alex Sanchez-Stern, James R Wilcox, and Zachary Tatlock. Automatically improving accuracy for floating point expressions. *Acm Sigplan Notices*, 50(6):1–11, 2015.
- [40] Louis-Noël Pouchet et al. Polybench: The polyhedral benchmark suite. <https://github.com/MatthiasJReisinger/PolyBenchC-4.2.1>, 437:1–1, 2012.
- [41] Louis-Noël Pouchet, Emily Tucker, Niansong Zhang, Hongzheng Chen, Debjit Pal, Gabriel Rodríguez, and Zhiru Zhang. Formal verification of source-to-source transformations for hls. In *Proceedings of the 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, pages 97–107, 2024.
- [42] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. *Acm Sigplan Notices*, 48(6):519–530, 2013.

- [43] KC Shashidhar, Maurice Bruynooghe, Francky Catthoor, and Gerda Janssens. Verification of source code transformations by program equivalence checking. In *Compiler Construction: 14th International Conference, CC 2005, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2005, Edinburgh, UK, April 4-8, 2005. Proceedings 14*, pages 221–236. Springer, 2005.
- [44] Gus Henry Smith, Andrew Liu, Steven Lyubomirsky, Scott Davidson, Joseph McMahan, Michael Taylor, Luis Ceze, and Zachary Tatlock. Pure tensor program rewriting via access patterns (representation pearl). In *Proceedings of the 5th ACM SIGPLAN International Symposium on Machine Programming, PLDI '21*. ACM, June 2021.
- [45] Atefeh Sohrabizadeh, Cody Hao Yu, Min Gao, and Jason Cong. Autodse: Enabling software programmers to design efficient fpga accelerators. *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, 27(4):1–27, 2022.
- [46] Ross Tate, Michael Stepp, Zachary Tatlock, and Sorin Lerner. Equality saturation: a new approach to optimization. In *Proceedings of the 36th annual ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 264–276, 2009.
- [47] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19, 2019.
- [48] Ecenur Ustun, Ismail San, Jiaqi Yin, Cunxi Yu, and Zhiru Zhang. Impress: Large integer multiplication expression rewriting for fpga hls. In *2022 IEEE 30th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, pages 1–10. IEEE, 2022.
- [49] Ecenur Ustun, Cunxi Yu, and Zhiru Zhang. Equality saturation for datapath synthesis: A pathway to pareto optimality. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*, pages 1–2. IEEE, 2023.
- [50] Hrishikesh Vaidya, Akilesh B, Abhishek Patwardhan, and Ramakrishna Upadrasta. Polybench-nn. <https://github.com/IITH-Compilers/PolyBench-NN>, 2018.
- [51] Alexa VanHattum, Rachit Nigam, Vincent T Lee, James Bornholt, and Adrian Sampson. Vectorization for digital signal processors via equality saturation. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 874–886, 2021.
- [52] Sven Verdoolaege, Gerda Janssens, and Maurice Bruynooghe. Equivalence checking of static affine programs using widening to handle recurrences. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 34(3):1–35, 2012.
- [53] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. Egg: Fast and extensible equality saturation. *Proceedings of the ACM on Programming Languages*, 5(POPL):1–29, 2021.
- [54] Jackson Woodruff, Thomas Koehler, Alexander Brauckmann, Chris Cummins, Sam Ainsworth, and Michael FP O’Boyle. Rewriting history: Repurposing domain-specific cgras. *arXiv preprint arXiv:2309.09112*, 2023.
- [55] Nan Wu, Yuan Xie, and Cong Hao. Ironman: Gnn-assisted design space exploration in high-level synthesis via reinforcement learning. In *Proceedings of the 2021 on Great Lakes Symposium on VLSI*, pages 39–44, 2021.
- [56] Yichen Yang, Phitchaya Phothilimthana, Yisu Wang, Max Willsey, Sudip Roy, and Jacques Pienaar. Equality saturation for tensor graph superoptimization. *Proceedings of Machine Learning and Systems*, 3:255–268, 2021.
- [57] Hanchen Ye, Cong Hao, Jianyi Cheng, Hyunmin Jeong, Jack Huang, Stephen Neuendorffer, and Deming Chen. Scalehls: A new scalable high-level synthesis framework on multi-level intermediate representation. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 741–755. IEEE, 2022.
- [58] Jiaqi Yin, Zhan Song, Chen Chen, Qihao Hu, and Cunxi Yu. Boole: Exact symbolic reasoning via boolean equality saturation. *Design Automation Conference (DAC)*, 2025.
- [59] Cunxi Yu, Mihir Choudhury, Andrew Sullivan, and Maciej Ciesielski. Advanced datapath synthesis using graph isomorphism. In *2017 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 424–429. IEEE, 2017.
- [60] Yihong Zhang, Yisu Remy Wang, Oliver Flatt, David Cao, Philip Zucker, Eli Rosenthal, Zachary Tatlock, and Max Willsey. Better together: Unifying datalog and equality saturation. *Proc. ACM Program. Lang.*, 7(PLDI), jun 2023.
- [61] Ruizhe Zhao, Jianyi Cheng, Wayne Luk, and George A Constantinides. Polsca: Polyhedral high-level synthesis with compiler transformations. In *2022 32nd International Conference on Field-Programmable Logic and Applications (FPL)*, pages 235–242. IEEE, 2022.